

docs/strategy\_vehicle\_verX.md

# Strategy\_Vehicle\_verX / VehicleNegotiatorX 詳細解説

対象実装:

- MODEL3/Strategy\_Vehicle\_verX.py (class Vehicle(Vehicle\_BASE))
- MODEL3/Vehicle\_NegotiatorX.py (class VehicleNegotiatorX)

この文書は、StrategyX (verX) が実装している「交渉戦略」をコードの挙動に即して整理した解説です。

- タスク価値 (Qスコア) にもとづく放出候補の順序付け
- 交渉相手の選定 (空間×時間の近傍探索)
- 交渉中の提案列と受諾判定 (VehicleNegotiatorX)
- 署名 (実行候補の選別) の基準

関連:

- MODEL3/Vehicle.py (Vehicle\_BASE と RL プランナーのスコア関数)
- MODEL3/VRPTW\_functions.py (近傍探索: find\_vehicles\_in\_neighboring\_areas / find\_time\_zone / find\_vehicle\_by\_id)
- MODEL3/bulletin\_are\_search.py (calculate\_dynamic\_area)
- MODEL3/Negotiator.py (SAO 交渉の実行)

---

## 0. 前提: Q スコアと掲示板

### 0.1 Q スコア (標準化済み)

Strategy\_Vehicle\_verX は、Vehicle\_BASE.standardized\_q\_task\_scores(...) を用いて標準化された Q スコアを算出します。

- 実体は route\_planner.standardized\_discounted\_q\_task\_scores(...)
- 値は 各ステップでの feasible action の平均/分散を用いて正規化されたスコア
- 返り値は task.id -> float の辞書

- ルートプランナー未設定や例外時は `None` を返し、StrategyX 側で 0.0 にフォールバックします

## 0.2 掲示板 (Bulletin)

交渉相手の近傍探索に Bulletin の以下を参照します。

- `area_board`: 時間帯ごとの滞在エリア
  - `time_board`: `departure_time`, `return_time` など
  - `zones`, `X`, `n`: 時間帯 / 空間セル分割のパラメータ
- 

# 1. タスク価値スコアの使い方

## 1.1 スコア取得: `_task_value_scores(...)`

`Strategy_Vehicle_verX.Vehicle` は以下でタスクのスコアを取得します。

- 入力: `tasks` (省略時は `self.tasks`)
- 出力: `task.id -> score`
- `route_planner` が無い場合は **すべて 0.0** として扱う

## 1.2 並び順: `_tasks_sorted_by_value()`

スコアの **\*\*昇順** (低い値が先) **\*\***でタスクを並べます。

```
sorted(self.tasks, key=lambda task: scores.get(task.id, 0.0))
```

結果として、スコアの低いタスクから順に提案される動きになります。

## 1.3 ランク比: `_task_value_rank_ratio(...)`

スコアを **降順**に並べた順位から比率を計算します。

```
rank = 1 + (# scores > target_score)
rank_ratio = rank / N
```

- `rank_ratio` が **小さいほど上位** ( $1/N$  が最大スコア)
  - `rank_ratio <= 0.3` を「上位 30%」として扱う
-

## 2. 交渉提案生成 offer\_on\_negotiation(...)

### 2.1 全体の流れ

StrategyX は タスク数に関係なく、全タスクを対象に提案を作ります (ver1/verZ の K-means 分岐は無し)。

```
ordered_tasks = tasks sorted by Q (low -> high)
for task in ordered_tasks:
    area = calculate_dynamic_area(task.x, task.y, X, n)
    zone = find_time_zone(task.ready_time, zones)
    neighbors = find_vehicles_in_neighboring_areas(zone, area, area_board)
    neighbors += find_available_vehicles(time_board, task.ready, task.due)
    neighbors -= {self.id}
    for each neighbor:
        Offer(self -> neighbor, task)
```

### 2.2 近傍探索 (空間×時間)

- 空間: calculate\_dynamic\_area(...) でタスク座標をセル化
- 時間: find\_time\_zone(...) でタスクの ready\_time をゾーンに変換
- find\_vehicles\_in\_neighboring\_areas(...) が「近傍エリアにいる車両ID」を返します

### 2.3 find\_available\_vehicles(...) の条件

時間板の条件は以下で、タスクの時間窓と完全に前後でズレている車両を返します。

```
departure_time > task_due_date
OR
return_time < task_ready_time
```

名前 (available) と条件の直感は逆に見えますが、実装はこの判定です。

### 2.4 実装上の注意 (ID/オブジェクトの混在)

find\_available\_vehicles(...) の返り値は 車両オブジェクトですが、find\_vehicle\_by\_id(...) は 車両IDを前提に検索します。そのため、vehicles\_in\_neighbors に車両オブジェクトが混在すると 該当車両が見つからず Offer が生成されないケースが発生します。

### 3. 交渉前の優先順位 `before_negotiation()`

`before_negotiation` は `self.over_task` を Q スコア昇順で更新します。

- `self.over_task = self._tasks_sorted_by_value()`
- `make_neg_agent(...)` で `VehicleNegotiatorX.remove_list` に渡される

ただし `VehicleNegotiatorX` は `propose()` の最初のステップで `make_remove_list(taskA)` を呼ぶため、実際の `remove_list` は再計算される点に注意が必要です。（`before_negotiation` で渡した順序は初期値としてのみ機能）

---

### 4. 署名（実行候補の選別） `sign_contracts(list: list[Agree])`

`StrategyX` の署名は Q ランクのみで判定します。

#### 4.1 判定ロジック

各 `Agree` について `agreement.taskA` を評価対象とし、

1. `candidate_tasks = self.tasks (+ taskA if not in tasks)`
2. `scores = _task_value_scores(candidate_tasks)`
3. `rank_ratio = _task_value_rank_ratio(taskA.id, scores)`
4. `rank_ratio <= 0.3` なら署名

#### 4.2 特徴

- `taskB` やコスト差分は一切評価しない
  - `taskA` が上位 30% なら受諾、そうでなければ拒否
  - `route_planner` が無い場合はスコアが 0 になり、
    - `rank_ratio = 1/N` (N はタスク数)
    - `N >= 4` なら多くの合意が通りやすい
- 

### 5. `VehicleNegotiatorX` の交渉ロジック

`VehicleNegotiatorX` は `VehicleNegotiator` を継承し、提案生成はベースのまま、受諾判定と `remove_list` 生成だけを Q スコアベースで置き換えています。

## 5.1 提案 (propose) はベース実装

VehicleNegotiator の動きに従います。

- A 側: {"taskA": self.task\_a, "taskB": None} を固定提案
- B 側: remove\_list から taskB を 1 件ずつ提示 ("0" は taskB=None)

## 5.2 remove\_list 生成 (make\_remove\_list)

VehicleNegotiatorX.make\_remove\_list(taskA) は以下の順で構成します。

1. scores = \_task\_value\_scores(self.tasks) を取得
2. scores が無い場合は 現行ルート順のまま remove\_list を構成
3. scores がある場合は Q 昇順で並べ替え
4. 末尾に "0" を追加 (taskB=None の提案)

結果として B 側は Q の低いタスクから順に差し出す提案列になります。

## 5.3 受諾判定 (respond)

受諾判定は taskA の Q ランクだけで行われます。

1. taskA が無い場合は REJECT\_OFFER
2. 容量制約: taskA が未保持かつ  $\text{current\_weight} + \text{taskA.weight} > \text{max\_weight}$  なら REJECT\_OFFER
3. candidate\_tasks = self.tasks (+ taskA if not in tasks)
4. scores = \_task\_value\_scores(candidate\_tasks) が無い場合は REJECT\_OFFER
5. ランク比で応答:
  - rank\_ratio  $\leq 0.3 \rightarrow$  ACCEPT\_OFFER
  - rank\_ratio  $\leq 0.7 \rightarrow$  REJECT\_OFFER
  - rank\_ratio  $> 0.7 \rightarrow$  END\_NEGOTIATION

注意:

- taskB は評価に使われません (交換条件ではなく taskA の価値のみ)
- 時間窓やスラック等のルート可否判定は行いません

## 6. StrategyX の特徴と注意点

- **Q スコア中心:** 交渉・署名ともに Q ランクを主軸に判断し、コスト差分や距離は見ません
- **ルートプランナー依存:** route\_planner が無いと VehicleNegotiatorX の受諾が常に REJECT になりやすい
- **taskA 偏重:** 署名・受諾ともに taskA のみを見るため、taskB の影響は無視されます
- **ID/オブジェクト混在:** 近傍候補にオブジェクトが混ざると Offer が生成されない可能性があります